

28 | 追本溯源：从第一版 React Native 开始学会读懂源码

2022-9-1 蒋宏伟



你好，我是蒋宏伟。

作为一名程序员，你是否有过看底层源码深入学习 React Native 新架构冲动？但冲动过后，你是否又因为源码太多、太难，而放弃了呢？

的确，React Native 从 2015 年开始开源，至今已经 8 年了，经历了 70 个版本的迭代，最新的新架构源码，代码量极大、功能模块极多、模块之间的关系也极其复杂。但其实只要我们掌握正确的方法，读源码远没有你想象的那么难。

今天，我就给你介绍我常用的读源码的三个方法，分别是“时光机”、“找线头”和“鸟瞰图”，并以第一版 React Native 源码为例，教你如何通过读源码，一步一步理解 React Native 新架构。

时光机

我的第一招叫做“时光机”。“时光机”是什么意思呢？

React Native 官方的每篇博客都有发表时间，每行代码都有 git 的版本记录，我把阅读过去某个时间节点的文章和代码的方式，叫做坐着“时光机”去学习。

React Native 项目，经历了 8 年的开源和发展，从 0.1.0 版本迭代到 0.70.0，已经从一棵只有主干的小树苗，长成了有多根树枝的大树了。现在的新架构就包括了 JSI、Fabric、TurboModule、CodeGen、Herems、Metro、Yago 等模块。

那么，新架构这么多个模块，哪些模块是必不可少的核心模块，这些模块之间又是如何相互协作的呢？

要回答这些问题，我们不妨先乘坐“时光机”，去看看 React Native 历史上最重要的几个节点，这样就能把上面的枝叶都看清楚了。

我认为，React Native 最关键的三个节点分别是开源的第一版、跨端版本，以及即将到来的新架构版本。

React Native 开源的第一版官方也是有介绍的。从第一篇官方博客 [《React Native: Bringing modern web techniques to mobile》](#) 中，我们可以看出 React Native 立项的最初目的并不是跨端，而是要把现代 Web 技术带到移动端领域，包括 React 声明式的编程思想、浏览器和 W3C 的标准，还有 JavaScript 工具链。这样一来还可以吸引庞大的前端从业人员，前端开发者也能借助 React Native 开发 iOS 移动端。

第二个是跨端版本，跨端这个重要节点始于 2015 年 9 月的官方博客，[《React Native for Android: How we built the first cross-platform React Native app》](#)。在这篇博客中，官方首次提到了 React Native 的跨端能力。我个人用的第一个线上版本是 2016 年 6 月出来的 0.28 版本，这时候跨端技术就已经成熟了。

最新版本，截至现在，也就是 2022 年 9 月的新架构预览版。预览版已经集成了新架构的代码了，Facebook 系的 App 中也已经有上千个页面用上了新架构，我们也可以手动开启新架构。官方也写了 [《The New Architecture》](#) 和 [《Architecture Overview》](#) 两篇文章对新架构进行了介绍，介绍了为什么我们需要新架构、新架构的优势又是什么，以及如何迁移。

以上这几篇官方博客，我建议你按照时间顺序读一读，重点理解 React Native 团队的设计初衷。

看完官方博客之后，接着我们就要读源码了。当然今天这一讲，我们不可能把这几个重要版本的源码都读完，但我会给你介绍我读源码的方法。

我的建议是先读第一版的源码，再找几个自己熟悉的版本的源码读一读，最后再读新架构的源码。

为什么要这样呢？你可以先试想一下，如果我们先读新架构预览版的源码会发生什么？

我统计了一下 React Native 0.70 版本的代码量，一共有 3,840 个文件，581,562 行代码。假设你技术出众，一小时能读 1000 行代码，一天能读 12 小时，那么读完核心库的 581,562 行代码，你也要 48 天。

更何况，React Native 新架构中涉及 JavaScript、C++、Java、Objective-C 多种语言，又涉及前端、移动端、编译、布局、引擎多个领域知识，对技术广度也有很高的要求，遇到不懂的知识，你还得去 Google 翻文章系统地去学习，才能读懂。

所以说，直接读新架构源码的难度是非常高的。

我用 VS Code Counter 统计了 React Native 0.70 版本的代码明细和React Native 2015年1月30日开源的第一版代码统计明细，你可以看下其中的差异：

Total : 3840 files, 423989 codes, 80163 comments, 77410 blanks, all 581562 lines						Total : 344 files, 24303 codes, 8004 comments, 4760 blanks, all 37067 lines					
Summary / Details / Diff Summary / Diff Details						Summary / Details / Diff Summary / Diff Details					
Languages						Languages					
language	files	code	comment	blank	total	language	files	code	comment	blank	total
JavaScript	1,118	193,358	33,683	24,533	251,574	JavaScript	208	15,384	6,554	2,704	24,642
C++	1,221	84,712	22,522	21,684	128,918	Objective-C	55	6,694	864	1,364	8,922
Java	787	76,853	15,740	15,808	108,401	C++	58	906	452	504	1,862
Objective-C++	165	27,436	2,755	5,868	36,059	C	1	578	133	107	818
Objective-C	205	25,552	2,863	5,679	34,094	JSON	12	301	0	4	305
Ruby	80	4,704	878	1,027	6,609	Markdown	2	176	0	49	225
Markdown	24	4,145	8	1,453	5,606	XML	3	123	0	3	126
Shell Script	52	1,846	560	544	2,950	CSS	1	88	0	7	95
YAML	15	1,704	166	160	2,030	HTML	1	26	0	5	31
Groovy	7	1,164	520	227	1,911	Ini	1	21	0	9	30
XML	115	1,067	310	253	1,630	YAML	1	3	0	1	4
JSON	31	865	64	20	949	Shell Script	1	3	1	3	7
Batch	4	215	0	68	283						
Makefile	2	151	39	29	219						
Properties	10	73	38	27	138						
Ini	1	52	5	21	78						
HTML	2	52	0	3	55						
XSL	1	40	12	6	58						

阅读 React Native 新架构的源码也是如此，我们可以乘坐“时光机”，回到 React Native 开源的第一个版本。

你可以看到，**React Native 第一个版本的复杂度**，比现在的 0.70 版本低了一个量级。从文件数量上看，0.70 版本有 3840 个，第一版只有 344 个；从代码行数上看，0.70 版本有 581, 562 行，第一版只有 37,067 行；假设一天读 12,000 行，读完的天数也由 48 天，减少到了 3 天。

而且，React Native 第一版，基本上都是 JavaScript 和 iOS 相关的知识。

如果你是前端同学，遇到不懂的 iOS 相关知识，可以边看源码边 Google 一下补一补，不用精通，能够大致理解 iOS 的相关代码就行了。如果你是 iOS 或者 Android 同学，也可以用类似的方法进行源码阅读。

所以我同样建议你，通过“时光机”，回到 React Native 开源的第一个版本，以此方法来阅读 React Native 新架构的源码。

找线头

即便如此，但当你面对第一版的 344 个文件时，你是不是也会很迷茫，不知从哪个文件开始看呢？

这就要用到，我今天和你分享的第二个学习方法“找线头”。打个比方，收纳柜里的数据线乱做一团，从哪里开始收拾呢？当然是找到数据线的线头，一根根地把这些数据线收拾好。

学习源码的时候，咱们也得先找到源码的线头。

首先，第一步是将代码切换到第一版的源码。

你只需要敲 3 行命令即可：

```
$ git clone git@github.com:facebook/react-native.git
$ git log --reverse
```

```
commit a15603d8f1ecdd673d80be318293cee53eb4475d
Author: Ben Alpert <balpert@fb.com>
Date: Thu Jan 29 17:10:49 2015 -0800
```

```
Initial commit
```

```
$ git checkout a15603d
```

也就是，先在 GitHub 上克隆 React Native 仓库，然后按倒叙顺序查看 commit 的提交。你可以看到，React Native 第一版的提交时间是 2015年1月29日，提交校验值是 a15603d，执行 `git checkout a15603d` 即可切换到第一版。

第二步，就是观察项目的目录结构。

切换到第一个版本后，打开 react-native 项目，你会看到一个这样的项目结构：

```
.
├── Examples
│   ├── Movies
│   ├── TicTacToe
│   └── UIExplorer
├── Libraries
│   ├── BatchedBridge
│   ├── Bundler
│   ├── Components
│   ├── Device
│   ├── Fetch
│   ├── Interaction
│   ├── JavaScriptAppEngine
│   ├── RKBackendNode
│   ├── ReactIOS
│   ├── StyleSheet
│   ├── Utilities
│   ├── XMLHttpRequest
│   ├── react-native
│   └── vendor
├── ReactKit
│   ├── Base
│   ├── Executors
│   ├── Layout
│   ├── Modules
│   ├── ReactKit.xcodeproj
│   └── Views
├── jestSupport
└── packager
```

这里主要包括 6 个部分：

Examples;
Libraries;
packager;
ReactKit;
jestSupport;
配置文件。

这个项目结构就是创始团队对 React Native 最初构思，也是 React Native 的架构划分基础。其中 jestSupport 目录是用来支持 jest 单元测试的，packager 目录是对 javascript 代码进行编译和打包的工具，package.json、.eslintrc 等最外层的文件都属于配置文件，但这三部分并不是 React Native 架构的重点，剩下的 3 个部分才是 React Native 架构的精华。

React Native 架构整体上分为三层， Example 是业务、Libraries 是封装的库、ReactKit 是原生实现。

首先，Example 放置的示例代码的目录，从易到难包括三个示例，分别是TicTacToe 井字棋示例、Movies 电影列表页示例和UIExplorer 基础组件用法示例。创始团队想通过这三个示例，告诉我们怎么使用 React Native 去开发原生应用。

在 TicTacToe、Movies、UIExplorer 这三个项目中其实包含了一个线头，也就是它们的 main 函数：

```
#import <UIKit/UIKit.h>
#import "AppDelegate.h"

int main(int argc, char * argv[]) {
    @autoreleasepool {
        return UIApplicationMain(argc, argv, nil, NSStringFromClass([AppDelegate class]));
    }
}
```

main 函数是 iOS App 的入口函数，当 App 启动的时候，首先调用的就是 main 函数。然后再一层层往下调用，从 main 函数，到 UIApplicationMain 函数，再触发 AppDelegate 类上的生命周期。这块后面我们会有更详细的介绍。

接下来，我们看第二个核心目录。**第二个核心目录是 Libraries 目录，该目录中放置的全部都是 JavaScript 代码。**

Libraries 目录中的线头是 Libraries/react-native/react-native.js 文件，它的代码示例如下：

```
/* Libraries/react-native/react-native.js */
var React = require('React');

var ReactNative = {
  ...React,
  Bundler,
  Text,
  View,
  StyleSheet,
  /* ... */
};

module.exports = ReactNative;
```

你可以看到，react-native.js 文件不仅是 Libraries 目录线头，也是 React Native 框架 JavaScript 部分的导出文件。

在第一版中，ReactNative 是直接依赖 React 的，该 React 一部分是 Web 和 React Native 的共享的能力，一部分是为 React Native 定制的能力。

共享的能力包括咱们常用的 ReactComponent、ReactElement 等等；为 React Native 定制的能力，包括一些涉及宿主环境对象的操作，比如管理视图的 UIManager，它起到的作用类似于 Web 中的 DOM 操作。

你还可以看到，启动 App JavaScript 部分的 Bundler 类、React Native 封装好的视图组件 View 和文本组件 Text，以及管理样式的 StyleSheet 等等。

第三个核心目录是最底层的 ReactKit 目录，它放置的全部都是 iOS 的代码。

ReactKit 的命名风格，很明显模仿的是 iOS UIKit 的命名风格，它的意思是里面放的是为 React 编写的代码套件。其中，Executors 目录负责执行上层的 JavaScript 代码，Base 目

录负责实现 JavaScript 和 iOS 通信的 Native 部分，Views、Modules、Layout 目录负责将 iOS 组件、接口暴露给 JavaScript，并实现了布局功能。

单独阅读 ReactKit 目录，你很难找到一个明显的线头，那就只能从 main 函数和 react-native.js 文件两个线头中选一个线头开始源码阅读了。

选哪个线头呢？显然从 main 函数开始更好。

因为mian 函数掌管 App 启动，react-native.js 掌管 React Native 框架 JavaScript 部分启动。App 启动在先，React Native 框架 JavaScript 部分的执行在后，因此咱们从 main 函数开始读源码。

我们以井字棋游戏 Examples/TicTacToe/main.m 文件为例。应用程序启动时，首先会调用 main.m 文件中的 main 函数，接着 main 函数会调用 UIApplicationMain 函数创建一个应用程序对象，此时应用程序启动。

在应用程序启动即将完成时，就会触发 AppDelegate.m 文件中的 `application:didFinishLaunchingWithOptions:` 方法。这时，请你打开 AppDelegate.m 文件，找到相关的 `didFinishLaunchingWithOptions` 方法，其代码摘要如下：

```
// 在 iOS 应用启动完成后会触发 application didFinishLaunchingWithOptions 回调
- (BOOL)application:(UIApplication *)application didFinishLaunchingWithOptions:(NSDictionary {

    // React Native 应用根视图的初始化
    RCTRootView *rootView = [[RCTRootView alloc] init];

    // React Native 应用的 JavaScript 代码地址
    jsCodeLocation = [NSURL URLWithString:@"http://localhost:8081/TicTacToeApp.bundle"];

    // 将 JavaScript 代码挂到根视图上
    rootView.scriptURL = jsCodeLocation;

    // iOS 应用 UIWindow 主窗口初始化
    self.window = [[UIWindow alloc] initWithFrame:[UIScreen mainScreen].bounds];
    UIViewController *rootViewController = [[UIViewController alloc] init];

    // 将 React Native 应用根视图挂到 UIWindow 上，此时 React Native 视图展示在手机上了
    rootViewController.view = rootView;
```



```
self.window.rootViewController = rootViewController;  
}
```

在程序启动即将完成的回调中，React Native 的根视图 rootView 会先完成初始化。初始化之后，此时 rootView 还是一个空视图，什么都没有。紧接着，会执行 `rootView.scriptURL = jsCodeLocation` 这一行代码，这一行代码非常关键，它会执行打包后的 JavaScript 代码，也就是上述示例中的“<http://localhost:8081/TicTacToeApp.bundle>”文件。

执行完成 `rootView.scriptURL = jsCodeLocation` 这一行代码后，rootView 就由一个空视图，变成一个有着 TicTacToeApp 井字棋游戏内容的视图了。

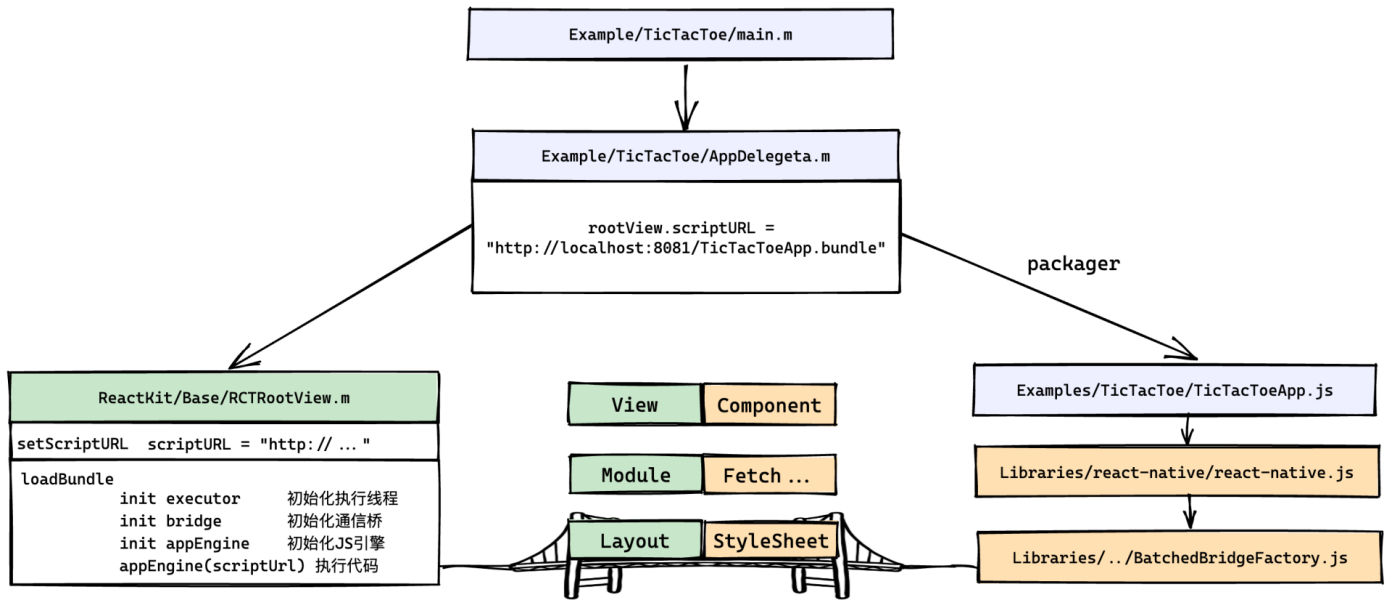
最后，先初始化 iOS 应用 UIWindow 主窗口，并将 React Native 应用根视图 rootView 挂到 UIWindow 上，此时 React Native 井字棋游戏就展示在手机上了。

鸟瞰图

找到线头后，接下来我们要理清的是，React Native 团队从整体上，是如何实现“Bringing modern web techniques to mobile”的目标。

我将整体理解作者实现方案的这一步，称之为画“鸟瞰图”，也就是从高处向下看，全面地、概括地理解 React Native 第一版中的 3 个核心目录的功能和联系。

React Native 第一版架构的鸟瞰图我已经为你画好了，你可以用它来帮你加深对 React Native 架构的理解。



在“找线头”这一步中，我们留了一个悬念，也就是 `rootView.scriptURL = “http://localhost:8081/TicTacToeApp.bundle”` 这一行代码究竟做了什么。

这一行代码，主要做了两件事情，第一件事是调用了 `ReactKit/Base/RCTRootView.m` 文件中的 `RootView` 类的 `setScriptURL` 方法，另一件事是请求以 `Examples/TicTacToe/TicTacToe.js` 文件为入口文件的脚本代码。

最重要的是 `RootView` 部分的实现。现在，请你打开 `ReactKit/Base/RCTRootView.m` 文件，其核心代码如下：

```

@implementation RCTRootView
{
    // 用于通讯的队列
    dispatch_queue_t _shadowQueue;
    // 通讯桥
    RCTBridge *_bridge;
    // JS引擎，底层是 JSCore
    RCTJavaScriptAppEngine *_appEngine;
    // 触控事件句柄
    RCTTouchHandler *_touchHandler;
}

// scriptURL 属性的 set 方法
- (void)setScriptURL:(NSURL *)scriptURL
{
    _scriptURL = scriptURL;
    // 调用 loadBundle 方法
    [self loadBundle];
}
  
```

```

}

// 加载 Bundle 代码
- (void)loadBundle
{
    // executor: 生成一个新的用于执行 JS 线程
    _executor = [[RCTContextExecutor alloc] init];
    // bridge: 处理 JS 和 Native 之间的相互通讯。
    // Native RCTXXX <=> native moduleIDs <==bridge==> message <=> js function
    _bridge = [[RCTBridge alloc] initWithJavaScriptExecutor:_executor
                                     shadowQueue:_shadowQueue
                                     javascriptModulesConfig:[RCTModuleIDs config]];

    // appEngine: JavaScriptCore
    _appEngine = [[RCTJavaScriptAppEngine alloc] initWithBridge:_bridge];

    // touchHandler: 绑定原生手势事件 + 用户触发时通过 bridge 通知 JS
    _touchHandler = [[RCTTouchHandler alloc] initWithEventDispatcher:_bridge.eventDispatcher r

    // 使用JS引擎, 执行 scriptURL 代码, 初始化所有的 Bundle 代码
    [_appEngine loadBundleAtURL:_scriptURL useCache:NO onComplete:callback];
}

```

上面这段代码有点长，如果你不熟悉 Objective-C 也没关系，我一步一步和你介绍。第一行代码 `@implementation RCTRootView` 的意思是，声明一个 `RCTRootView` 的类。`RCTRootView` 类有 4 个私有属性和两个主要的方法。

4 个私有属性分别是：

shadowQueue：用于通讯的队列；

bridge：通讯桥；

appEngine：JavaScript引擎，底层是 JavaScriptCore；

touchHandler：触控事件句柄。

两个方法分别是：

setScriptURL：也就是 scriptURL 属性的 set 方法；

loadBundle：加载 Bundle 代码的方法。

当给 `rootView.scriptURL` 设置值的时候，底层调用的是 `setScriptURL` 方法，该方法不仅会将“<http://localhost:8081/TicTacToeApp.bundle>”赋值给 `_scriptURL` 属性，还会接着调用 `loadBundle` 方法。

在 `loadBundle` 方法中，会依次初始化 `executor`、`bridge`、`appEngine`、`touchHandler` 四个属性。

`executor` 的作用是创建一个执行 `Bundle` 的线程，该线程也叫做 `JavaScript` 线程，是独立于 `UI` 主线程之外的线程。起两个线程的作用是，让 `Bundle` 代码和 `Native` 代码同时执行。如果把 `Bundle` 的执行和 `Native` 代码的执行放在同一个线程，而不是分别由两个线程执行，会有一个很明显的缺陷，二者会相互阻塞。

`bridge` 的作用是处理 `JavaScript` 和 `Native` 之间的相互通讯。你可以看到，初始化 `bridge` 时接收了 3 个参数，分别是 `executor`、`shadowQueue`、`RCTModuleIDs` 的 `config`。`bridge` 通过 `RCTModuleIDs` 维护了 `JavaScript` 函数和 `Native` 函数之间的映射关系，该映射关系是以字符串形式存在的。但为了应对频繁的相互调用，就需要把调用信息放到一个消息队列中，消息队列 `shadowQueue` 起到的作用是削峰平谷。同样，`bridge` 是在 `executor` 创建的 `JavaScript` 线程中执行 `Bundle` 代码的。

`appEngine` 底层就是 `JavaScriptCore` 引擎，`JavaScriptCore` 引擎的作用就是执行“<http://localhost:8081/TicTacToeApp.bundle>”文件的 `JavaScript` 代码。它会先执行 `TicTacToeApp.js` 入口文件的代码，并继续执行 `react-native.js` 框架源码，初始化 `JavaScript` 部分 `bridge` 功能。

`touchHandler` 的作用是，事先绑定原生手势事件，当用户点击屏幕触发手势事件时，再通过 `bridge` 通知 `JavaScript`。

最后一行代码的 `appEngine.loadBundleAtURL(scriptURL)` 的作用是正式执行 `Bundle` 代码。

`loadBundleAtURL` 调用完成后，所有的 `JavaScript` 和 `Native` 之间的桥梁已经搭建完成。

既然桥已搭好，那么剩下的将类 `Web` 的组件、接口、布局和 `Native` 的组件、接口、布局进行映射就顺理成章了。

大体上讲，React Native 上层抽象出来的 Libraries/Components 组件，底层对应的是 ReactKit/Views 组件；上层抽象出来的 Libraries/Fetch、Libraries/XHR 网络接口，其底层实现的是 ReactKit/Modules/RCTDataManager.m 文件；上层实现的样式 Libraries/StyleSheet，其底层是 ReactKit/Layout。

此外，React Native 还通过 packager 目录中提供的编译打包工具，让原生应用具备了快速调试和热更新的能力。

通过以上源码的学习，我们就了解了 React Native 团队，实现“Bringing modern web techniques to mobile”的整体思路。

总结

看源码是有方法的，我们可以先乘坐“时光机”回到原点，了解作者最初的设计思想和源码实现。然后，到源码中“找线头”，先找到整个程序的调用入口，然后一步一步地跟踪下去。最后再按照作者思路 and 实现，画一张架构的“鸟瞰图”，从整体上理解架构的实现原理。

以上三步，每走一步，我们都会将读源码这个事情变得更简单。从 0.70 新架构预览版的 3,840 个文件，再到第一次提交的 344 个文件，最后是“鸟瞰图”中梳理出的 10 多个核心文件或目录。

结合作者最初的设计思想和源码实现，我们能看出在第一版中，并没有 Android 代码，是因为 React Native 第一版并不是为跨平台设计的，而是为复用 Web 开发模式而设计的，让原生开发能够复用 Web、React 工具链。React Native 跨平台是在此之后才提出来的目标。最近的目标，才是近几年反复宣传新架构性能提升。

不同的理念，不同的架构，但前一代架构是后一代架构的基础，后一代架构是前一代架构的升华。

我们追根溯源，才能把握核心本质。

思考题

请你思考一下，为什么 React Native 不是一上来就使用 JSI，而是先用 Bridge 消息队列的形式实现了 JavaScript 和 Native 的双向通信呢？

欢迎在评论区留下你的想法，我们下节课见。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

上一篇 27 | 跨端的机遇：小程序、Flutter和React Native原理对比

下一篇 29 | 弄清现状：新架构预览版究竟长什么样？

精选留言 2